PROJECT REPORT

on

"Intelligent Academic Performance Tracker & Visualizer"

[*Python Programming Post-Lab*]

Submitted By:

Sahil Suryawanshi(CS24037)

Guided By:

Ms. Ashvini Tarar

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH, NAGPUR.

(An Autonomous Institute, Affiliated to RTMNU, Nagpur)

2025-2026

© S.B.J.I.T.M.R Nagpur 2026

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to RTMNU, Nagpur)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SESSION 2025-2026

CERTIFICATE

This is to certify that the Mini Project titled **"Intelligent Academic Performance Tracker & Visualizer"** is a bonafide work of **Sahil Suryawanshi** carried out for the partial fulfillment of course work of "**Python Programming (N- SECCS401P)**", 4th Semester, Bachelor of Technology in **Computer Science & Engineering, S.**B. Jain Institute of Technology, Management & Research, Nagpur.

Ms. Ashvini Tarar
Assistant Professor
(Project Guide)

Dr. Anup Gade
Head of Department

INDEX

CERTIFICATE		2
INDEX		3
LIST OF FIGURES		3
CHAPTER 1 INTRODUCTION 1		5
CHAPTER 2 METHODOLOGY 2		2
CHAPTER 3 TOOLS/PLATFORMS		3
CHAPTER 4	DESIGN & IMPLEMENTATION	5-9
	4.1 ALGORITHM	
	4.2 FLOWCHART	
	4.3 SOURCE CODE	
CHAPTER 5	RESULT & DISCUSSION	10-12
	5.1 OUTPUT	
	5.2 DISCUSSION	
	5.3 APPLICATION	
CHAPTER 6	CONCLUSION	13
	REFERENCES	13

LIST OF FIGURES

FIG. NO.	TITLE OF FIGURE	PAGE NO.
1	4.1 Flow chart	6
2	**5.1 Output**	9

CHAPTER 1
INTRODUCTION

The **Intelligent Academic Performance Tracker & Visualizer** is a data analysis system designed to monitor, evaluate, and present student academic performance in a meaningful way. It converts raw marks data into useful insights such as percentage, GPA, and performance trends. The main purpose of this system is to help teachers and institutions easily understand student performance, identify weak students, and make better academic decisions based on data rather than assumptions.

The system works by collecting student data stored in a CSV file, which includes details such as name, department, and marks in different subjects. This data is then processed using Python. One of the key steps in the system is data preprocessing, where missing values in the dataset are handled by replacing them with the average marks of the respective subject. This ensures that the dataset remains complete and suitable for analysis.

After cleaning the data, the system calculates important performance metrics. The percentage is calculated by taking the average of marks obtained in all subjects, and the GPA is derived by dividing the percentage by 10. These metrics provide a standardized way to evaluate and compare student performance. The system also identifies “at-risk” students, defined as those scoring below 40%, so that educators can take timely action to support them.

To make the analysis more understandable, the system uses data visualization techniques. A histogram is used to display the distribution of student percentages, helping to identify overall performance patterns. A bar chart is used to show department-wise average performance, allowing easy comparison between different departments. These visual tools make complex data easier to interpret.

The project is implemented using Python along with libraries like Pandas and Matplotlib. Pandas is used for data handling and analysis, while Matplotlib is used for creating graphical representations. The system is simple, efficient, and scalable, making it suitable for analyzing large datasets.

Overall, the Intelligent Academic Performance Tracker & Visualizer provides an effective solution for academic data analysis. It reduces manual effort, improves accuracy, and helps in better decision-making. Although it currently works with basic datasets and visualizations, it can be further enhanced by adding features like graphical user interfaces, real-time data integration, and predictive analysis using machine learning.

In conclusion, the Intelligent Academic Performance Tracker & Visualizer is a practical and efficient tool that demonstrates the power of data analysis in education. By transforming raw student marks into meaningful insights and visual representations, it enables educators to quickly assess performance, identify at-risk students, and compare departmental outcomes. The system not only saves time and reduces manual effort but also supports data-driven decision-making, ultimately contributing to improved academic performance and better student support strategies.

CHAPTER 2

METHODOLOGY

The development of this project follows a systematic and structured approach to ensure accurate analysis and visualization of student performance data.

1. Requirement Analysis

The main requirement of this project is to develop a system that can analyze and visualize student academic performance efficiently. The system should be able to read student data from a CSV file, handle missing values, calculate performance metrics such as percentage and GPA, and identify at-risk students who score below a defined threshold (40%). Additionally, the system should generate visual representations like histograms and bar charts to help in better understanding of performance trends.

2. System Design

The system is designed with three main components: input, process, and output. The input consists of a CSV file containing student details such as name, department, and subject marks. The processing stage includes data cleaning, handling missing values, calculating percentage and GPA, and grouping data for analysis. The output includes both textual results, such as lists of at-risk students, and graphical outputs like histograms for grade distribution and bar charts for department-wise performance comparison.

3. Development Process

The project is developed using Python programming language with the help of Pandas and Matplotlib libraries. Pandas is used for reading and processing the dataset, handling missing values, and performing calculations. Matplotlib is used to create visualizations. The development includes creating functions and logic to compute percentage, GPA, and identify at-risk students. The system is implemented in a simple and efficient manner, ensuring readability and ease of use.

4. Implementation Strategy

The implementation involves loading the dataset using Pandas and cleaning it by replacing missing values with the average of respective subjects. A list of subjects is defined, and the percentage is calculated by taking the mean of marks across these subjects. GPA is computed by dividing the percentage by 10. Students scoring below 40% are filtered as at-risk. Data grouping is used to calculate department-wise averages. Visualization is implemented using Matplotlib to generate a histogram for overall performance distribution and a bar chart for department comparison.

5. Testing and Debugging

The system is tested using sample student data to ensure correctness of calculations and outputs. The percentage and GPA calculations are verified manually for accuracy. The identification of at-risk students is checked against expected results. Visualizations are tested to ensure they correctly represent the data. Error handling is verified by testing cases such as missing values and incorrect file inputs. Debugging is performed to ensure smooth execution without runtime errors.

CHAPTER 3

TOOLS / PLATFORMS

The development of the Intelligent Academic Performance Tracker & Visualizer requires basic software tools and minimal hardware resources. These tools help in efficient data processing, analysis, and visualization of student performance.

3.1 SOFTWARE REQUIREMENT

The following software tools were used in this project:

1. Python Programming Language

Python is the primary programming language used to develop this project. It is simple, easy to learn, and widely used for data analysis and visualization tasks.

2. Pandas Library

Pandas is used for reading the CSV file, handling missing values, and performing calculations such as percentage and GPA efficiently.

3. Matplotlib Library

Matplotlib is used to create visualizations such as histograms and bar charts, helping in better understanding of student performance data.

4. Code Editor (VS Code)

Visual Studio Code (VS Code) is used to write and run the Python program. It provides features like syntax highlighting and debugging, making development easier.

5. Command Line / Terminal

The terminal is used to execute the Python script and display outputs including student performance and graphs.

3.2 HARDWARE REQUIREMENT

The hardware requirements for this project are minimal:

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: At least 500 MB free space
- Input Devices: Keyboard and Mouse
- Output Device: Monitor

3.3 PLATFORM USED

The system is platform independent and can run on operating systems like Windows, Linux, and macOS without any modification.

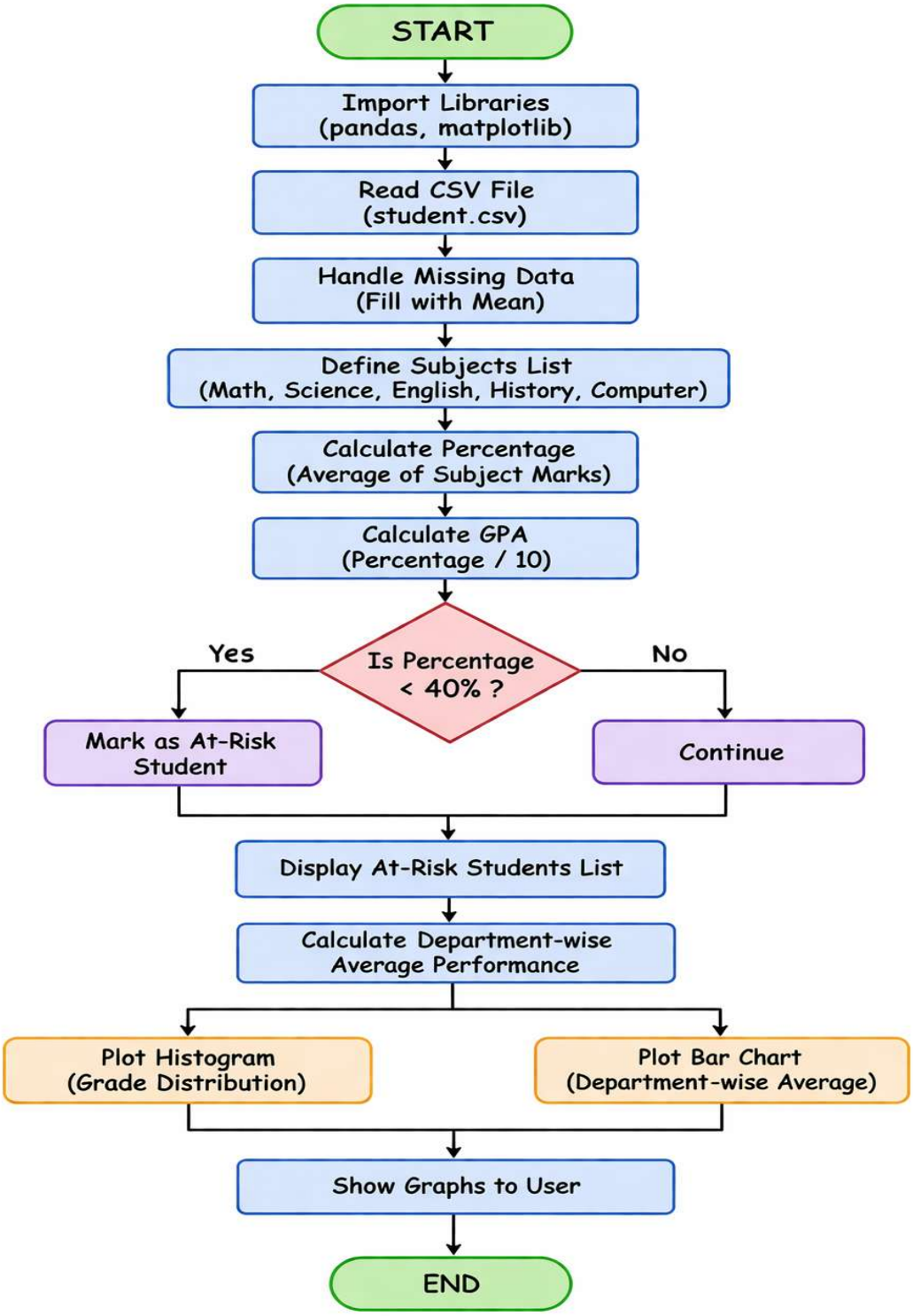
CHAPTER 4

DESIGN & IMPLEMENTATION

4.1 ALGORITHM

5 Begin the program
6 Import required modules (pandas, matplotlib)
7 Load the CSV file containing student data
8 Handle missing values by replacing them with the average of respective subjects
9 Define the list of subjects (Math, Science, English, History, Computer)
10 Calculate percentage by taking the average of subject marks for each student
11 Calculate GPA by dividing percentage by 10
12 Identify at-risk students whose percentage is below 40%
13 Display the list of at-risk students
14 Group data by department to calculate average performance
15 Generate histogram to show distribution of student percentages
16 Generate bar chart to display department-wise average performance
17 Display all graphs to the user
18 If any error occurs (file not found or invalid data), display an error message
19 End the program

4.2 FLOWCHART



4.3 SOURCE CODE

```
import pandas as pd
import matplotlib.pyplot as plt

# Load CSV file
df = pd.read_csv("student.csv")

# Fill missing values with subject average
df.fillna(df.mean(numeric_only=True), inplace=True)

# Subjects list
subjects = ['Math', 'Science', 'English', 'History', 'Computer']

# Calculate Percentage
df['Percentage'] = df[subjects].mean(axis=1)

# Calculate GPA (out of 10)
df['GPA'] = df['Percentage'] / 10

# Identify at-risk students (<40%)
at_risk = df[df['Percentage'] < 40]

print("\n=== At-Risk Students (Below 40%) ===")
print(at_risk[['Name', 'Department', 'Percentage']])

# ----------- PLOTS -----------

# Histogram: Grade Distribution
plt.figure()
plt.hist(df['Percentage'], bins=10)
plt.title("Grade Distribution")
plt.xlabel("Percentage")
plt.ylabel("Number of Students")

# Bar Chart: Department-wise Average
dept_avg = df.groupby('Department')['Percentage'].mean()

plt.figure()
dept_avg.plot(kind='bar')
plt.title("Department-wise Average Performance")
plt.xlabel("Department")
plt.ylabel("Average Percentage")
```

```
plt.tight_layout()
plt.show()
```

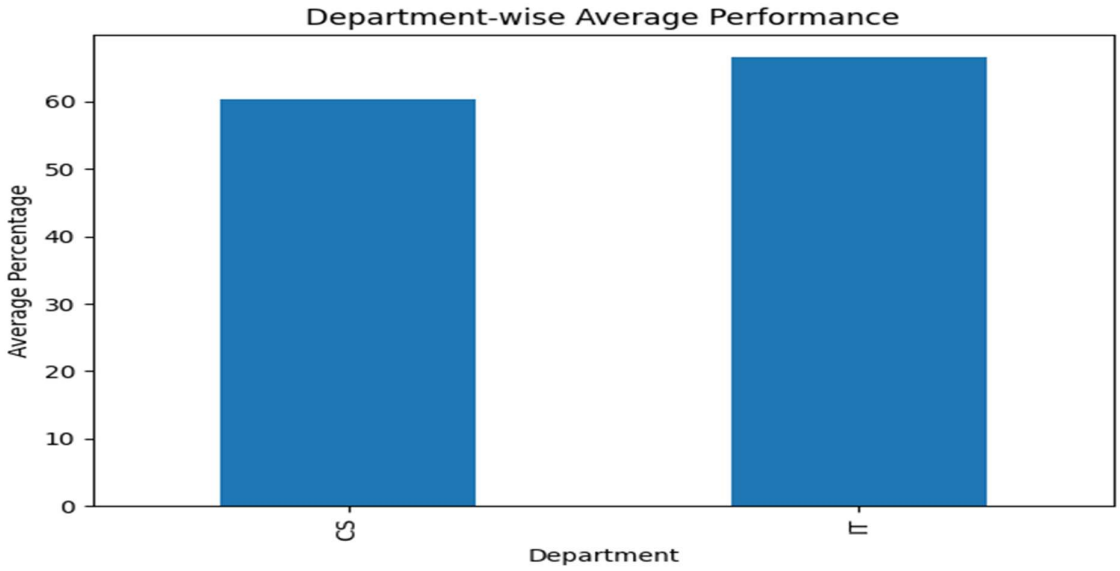
- Sample data:

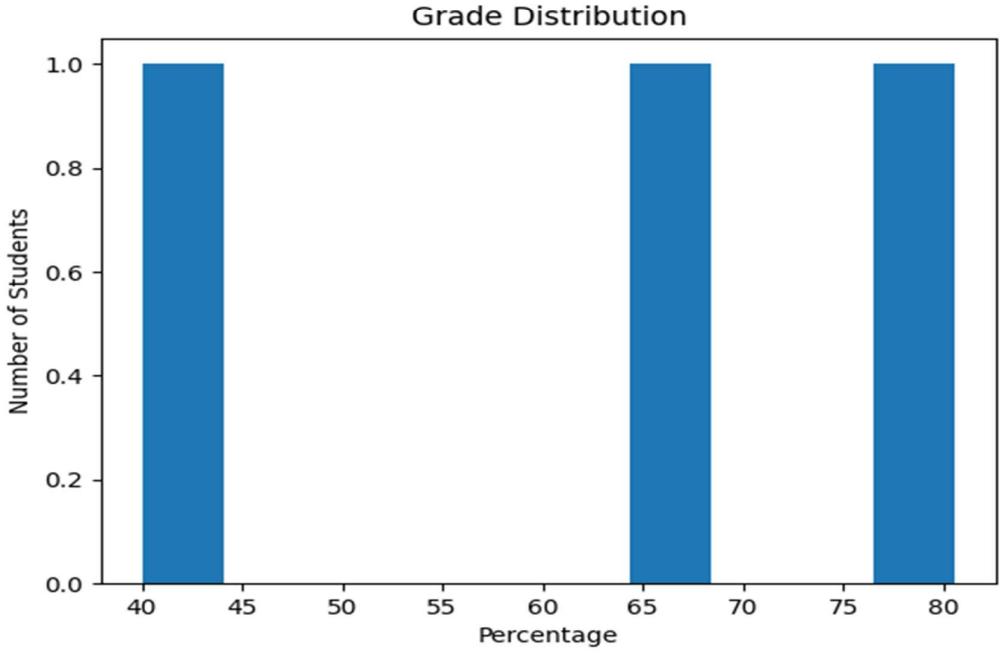
```
Name,Department,Math,Science,English,History,Computer
Rahul,CS,78,85,80,70,90
Amit,IT,60,,75,65,70
Sneha,CS,30,40,35,45,50
```

CHAPTER 5

RESULT & DISCUSSION

5.1 Output





5.2 DISCUSSION

The Intelligent Academic Performance Tracker & Visualizer developed using Python works effectively and fulfills all the basic requirements of an academic data analysis system. The application successfully performs core functions such as reading student data from a CSV file, handling missing values, calculating percentage and GPA, identifying at-risk students, and generating visualizations like histograms and bar charts, producing accurate results for all test cases.

During the execution of the project, it was observed that the use of Python libraries like Pandas makes reading and processing data simple and efficient. The program loads the dataset, fills missing values using subject averages, and processes each student's record systematically. This ensures that all data is handled correctly without any loss of information.

The percentage is calculated by averaging marks across all subjects, and GPA is obtained by dividing the percentage by 10. It was observed that these calculations are accurate and consistent. The system also identifies at-risk students scoring below 40%, helping in early detection and improvement of student performance.

Data visualization plays an important role in this project. The histogram displays the distribution of student percentages, while the bar chart shows department-wise average performance. These visualizations help in understanding patterns and comparing results effectively.

Error handling improves the reliability of the system. The program handles issues such as missing files, incorrect paths, and invalid inputs by displaying clear error messages and ensuring smooth execution without crashes.

This project highlights important concepts such as data manipulation using Pandas, visualization using Matplotlib, and structured programming techniques. It demonstrates how raw academic data can be converted into meaningful insights for better decision-making.

However, some limitations were observed during development. The system works only with CSV files and provides basic visualizations. It does not include advanced analytics or real-time updates. Despite these limitations, the project performs reliably and meets its intended purpose.

5.3 APPLICATION

The Intelligent Academic Performance Tracker & Visualizer has various practical uses in real-life situations where academic data analysis and performance monitoring are required. Some of the important applications are as follows:

1. **Student Performance Analysis**
   The system can be used by schools and colleges to analyze student marks and evaluate overall academic performance efficiently.
2. **Identification of At-Risk Students**
   It helps in identifying students who are scoring below the required threshold (40%), allowing teachers to take early corrective actions and provide additional support.
3. **Department Performance Comparison**
   The tool allows comparison of performance across different departments using graphical representations, helping institutions identify strong and weak areas.
4. **Educational Use**
   The project serves as a useful learning resource for students studying data analysis, as it provides practical experience with data handling, processing, and visualization.
5. **Decision Making Support**
   Teachers and administrators can use the insights generated by the system to make informed decisions regarding teaching strategies and academic improvements.
6. **Data Visualization Learning**
   The project helps users understand how graphical tools like histograms and bar charts can be used to represent data in a clear and meaningful way.
7. **Academic Data Management**
   The system provides a simple method to organize and manage student data stored in CSV files, making it easier to handle large datasets efficiently.

CHAPTER 6
CONCLUSION

The Intelligent Academic Performance Tracker & Visualizer was successfully designed and implemented using Python programming. The project demonstrates the effective use of data handling, analysis, and visualization techniques to build a functional academic performance monitoring system.

The application efficiently reads student data from a CSV file, processes it by handling missing values, and calculates key performance indicators such as percentage and GPA. It also successfully identifies at-risk students who score below 40% and provides accurate results for all test cases.

Through this project, important programming concepts such as data manipulation using Pandas, data visualization using Matplotlib, and structured programming were understood and applied practically. The use of graphical representations like histograms and bar charts enhances the understanding of student performance trends.

Although the project has some limitations such as working only with CSV files and providing basic visualizations, it serves as a strong foundation for understanding data analysis concepts. Future improvements can include advanced analytics, real-time data processing, and the development of a graphical user interface.

In conclusion, this project enhances programming and analytical skills and demonstrates how data can be transformed into meaningful insights, helping improve academic performance and decision-making in the education system.

REFERENCES

1. Python Official Documentation — https://docs.python.org
2. W3Schools Python Tutorials — https://www.w3schools.com/python
3. GeeksforGeeks — Pandas and Data Analysis — https://www.geeksforgeeks.org
4. Matplotlib Documentation — https://matplotlib.org
5. Stack Overflow — https://www.stackoverflow.com
6. GitHub Repository — https://www.github.com